Parallel Numerical Library

Debug report: what went wrong, and what each fault taught

Olajide Badejo

August 2026

Abstract

This is the engineering log of the project, kept from the first command and grouped here by theme. Each entry records the symptom as it presented, the root cause once found, the options considered, the fix and why it was chosen over the alternatives, and how it was verified.

It is a deliverable of equal rank to the main report because the main report contains only the conclusions, and the conclusions are not the part that is hard to reproduce. Several entries below describe faults that produced entirely plausible numbers, which is the failure mode worth documenting: an obvious crash teaches nothing, whereas a solver that converges in one iteration for the wrong reason will be believed.

Contents

Chapter 1

Environment and toolchain

Four of these five changed the structure of the build. They are first because they were first: the project could not compile until they were resolved.

1.1 ENV-01: nvcc rejects the project compiler, and its own default too

Symptom. Configuration failed inside `project(... CUDA)` with a compiler identification error. Invoking `nvcc` by hand on a trivial kernel with the system default host compiler failed differently, inside a system header:

```
/usr/include/x86_64-linux-gnu/c++/15/bits/c++config.h(586):
    error: expected a "("
      if consteval { return true; } else { return false; }
```

Root cause. Two separate problems presenting as one. First, CUDA 13.3 supports GCC only up to 15, and this project uses GCC 16 for the host code. Second, even at GCC 15, which nvcc claims to support, libstdc++ 15 uses the C++23 `if consteval` statement in `c++config.h`, and the EDG frontend inside nvcc cannot parse it. The nominally supported combination is also broken.

Options. Dropping the host code to GCC 14 throughout was rejected: it costs `<mdspan>` and drops the OpenMP level from 5.2 to 4.5, both of which the project uses. Passing `-allow-unsupported-compiler` was rejected because the failure is a genuine parse error in a system header, not a version check being cautious.

Fix. Compile only the `.cu` files with `nvcc -ccbin g++-14 -std=c++20`, and declare every CUDA entry point `extern "C"` taking plain pointers and scalars. No libstdc++ type crosses the boundary, so the two compilers never have to agree on a C++ ABI, only on the platform C ABI, which they do by definition. This also enforces at the ABI level the rule that backend files never leak their model's types.

Verification. A kernel built this way links against GCC 16 host code using `<mdspan>` and runs on the GPU returning the expected values.

1.2 ENV-02: a CMake setting that arrives too late

Symptom. After ENV-01, configuration still failed identically even though `set(CMAKE_CUDA_HOST_COMPILER g++-14)` was present in `CMakeLists.txt`.

Root cause. The line sat after `project(... LANGUAGES CXX CUDA)`. Compiler identification runs inside `project()`, so the setting arrived too late and identification used the default host compiler, which nvcc rejects.

Fix. Set it as a cache variable before `project()`, with a comment saying why so it cannot be tidied back down.

1.3 ENV-03: MPI version macros describe conformance, not capability

Symptom. The specification names MPI 5.0 as its reference and asks which of its features the installed library implements. `MPI_Get_version` returns 3.1 and the `MPI_VERSION` compile time macro is 3.

Root cause. OpenMPI 5.0.x does not claim conformance to MPI 4.0 as a whole, so it leaves the version macro at 3.1, but it ships many MPI 4.0 entry points. Link probes confirmed that `MPI_Session_init`, `MPI_Comm_create_from_group`, `MPI_Isendrecv`, `MPI_Allreduce_init`, `MPI_Info_create_env`, `MPI_Barrier_init` and `MPI_Comm_idup_with_info` all resolve.

Fix. Probe for symbols with `check_cxx_symbol_exists` rather than testing the version macro. Guarding on the macro would disable functionality that demonstrably works; assuming the functions are present would break on a stricter library. The core path uses only MPI 3.1 guaranteed API.

1.4 ENV-04: WSL2 hides the hybrid core topology entirely

Symptom. Objective 3 asks for the performance versus efficiency core knee on an i7-14700K, which has eight performance cores of two threads each and twelve efficiency cores. Inside the guest, `lscpu` reports fourteen uniform cores, every logical processor reports the same 2048K L2 and `cpu_capacity` of 1024, the hybrid flag is absent from `/proc/cpuinfo`, and `/sys/.../cpufreq` does not exist.

Root cause. WSL2 is a Hyper-V virtual machine and synthesises a homogeneous topology. Separately, `sched_setaffinity` binds a thread to a guest virtual processor and the hypervisor remains free to place that processor on any host core, so pinning is a hint about the guest and not a guarantee about silicon.

Options. Assuming the conventional enumeration, that host processors 0 to 15 are performance core threads and 16 to 27 are efficiency cores, was rejected: it is unverifiable from inside the guest and would produce confident labels with nothing behind them. Rebooting to bare metal was out of scope, the specification fixing WSL2 as the environment.

Fix. Measure rather than ask, and take the knee from a measurement that survives virtualisation. The topology probe times an identical kernel on each logical processor and reports a two group split only when the throughputs separate by a margin dominating the within group spread; otherwise it says so and the core type policies decline to bind. The knee is read from the aggregate scaling curve, which depends only on how many cores are engaged and not on which.

1.5 ENV-05: the CUDA host compiler's libstdc++ hijacks the link

Symptom. Enabling the CUDA backend broke a link that had worked throughout the project, with undefined references to `std::__detail::__wait_impl`, `std::__detail::__notify_impl` and `_M_setup_proxy_wait`, all reached from the jthread pool. Nothing in the CUDA code uses threads, and the failing objects were compiled by GCC 16.

Root cause. Those symbols are the out of line half of GCC 16's atomic wait and notify support, which `std::jthread` and `std::barrier` need. CMake detects the CUDA implicit link directories by asking nvcc, and nvcc drives g++-14, so the list contained `/usr/lib/gcc/x86_64-linux-gnu/14`. That `-L` landed ahead of the search path g++-16 adds for itself, so `-lstdc++` resolved to GCC 14's copy.

Two wrong turns before finding it. The first guess was that spaces in the project path were to blame, which a controlled test disproved. The second was that the link was driven by the wrong compiler, which setting the linker language did not fix because g++-16 was already driving it. The problem was the search order, not the driver.

Fix. Filter every version specific GCC directory out of the CUDA implicit link directories, which is safe precisely because the link is driven by the C++ compiler and it contributes its own. The configure step prints each directory it drops, so the filtering is visible.

Chapter 2

Numerics

These are the entries worth reading twice. Each produced a plausible number rather than an obvious failure.

2.1 NUM-01: the manufactured solution was an eigenvector

Symptom. The test asserting conjugate gradient needs $O(n)$ iterations on the model problem failed: the count did not grow at all between a 31 and a 63 grid. Measured directly, the method converged in exactly one iteration at every size.

Root cause. Not a bug in the method. The eigenvectors of the five point stencil are $\sin(p\pi x)\sin(q\pi y)$, and the manufactured solution $u = \sin(\pi x)\sin(\pi y)$ is exactly the lowest of them. With a zero initial guess the residual is a single eigenvector, so the Krylov subspace is one dimensional and the first step is exact. The problem was degenerate, not the method fast.

The same choice cuts the other way for the stationary methods: the slowest decaying mode of the Jacobi iteration matrix is that same lowest mode, so a single mode source isolates the asymptotic rate cleanly and reproduces the closed form spectral radius to six digits. One choice is ideal for one family and useless for the other.

Options. Changing to a polynomial such as $x(1-x)y(1-y)$ was rejected: its fourth derivatives vanish, so the five point stencil becomes exact for it and the second order accuracy test loses its subject. A sum of a few sine modes was rejected because it only moves the problem, a sum of three eigenvectors making the method terminate in three steps.

Fix. Provide both right hand sides and use each where it is honest. The manufactured sine source keeps the closed form solution and is used by the discretisation error and stationary rate tests; a seeded spectrally rich source excites the whole spectrum and is used for anything involving conjugate gradient and for the benchmark sweep, where a degenerate spectrum would flatter one method over the others.

Verification. With the rich source the count grows from 41 to 93 as the grid doubles, consistent with the $O(n)$ bound. A dedicated test now asserts the one iteration behaviour on the single mode source, so the trap stays recorded rather than merely avoided.

2.2 NUM-02: a default that was right for one solver and wrong for another

Symptom. Richardson reported divergence on both a 4 by 4 dense system and a 15 by 15 Poisson problem, while its own dedicated convergence test passed.

Root cause. The relaxation factor defaulted to one. The tests that passed set it explicitly to zero, meaning "ask the solver"; the tests that failed left the default. Richardson with $\omega = 1$ on the Poisson operator has iteration matrix $I - A$, whose eigenvalues reach $1 - 8 = -7$, so the spectral radius is seven and it diverges immediately. The default had been chosen thinking of SOR, where one means Gauss Seidel and is harmless.

Fix. The default is now zero, meaning each solver picks its own: Young's closed form optimum for SOR, one for SSOR, and the reciprocal of the Gershgorin bound for Richardson. Zero is the only default correct for every member of the family. The general lesson is that a shared options struct wants defaults meaning "ask the component", not defaults that suit whichever component was written first.

2.3 NUM-03: power iteration is the wrong way to choose a step

Symptom. The first Richardson implementation estimated the largest eigenvalue by power iteration and diverged even when asked to choose its own step.

Root cause. Two faults. The Rayleigh quotient approaches $\lambda_{\max}$ from below, so an unconverged estimate yields a step larger than $1/\lambda_{\max}$ and can leave the convergence interval entirely. Separately, the starting vector was written across the whole padded grid including the Dirichlet boundary ring, which corrupts the operator.

Fix. Replaced by a Gershgorin bound, which overestimates the spectral radius by construction, so its reciprocal is always strictly inside the convergence interval. It is exact for the stencil, free to compute, deterministic, and identical on every backend, none of which the power iteration was.

2.4 NUM-04: a tolerance below the arithmetic

Symptom. The discretisation error tests asked for a relative residual of 10^{-13} and stopped converging when the solver they used changed from Gauss Seidel to optimally relaxed SOR.

Root cause. Not a regression. Stationary iterations have a rounding limited residual floor, and over relaxation raises it because it amplifies rounding noise. Measured on this operator in double precision:

grid	forward Gauss Seidel	SOR at optimal omega	conjugate gradient
15 by 15	4.4e-15	9.5e-15	8.1e-17
31 by 31	1.7e-14	5.6e-14	8.6e-17
63 by 63	7.0e-14	2.9e-13	9.4e-17

The requested 10^{-13} sat below the floor at 63 by 63, so the iteration could never report convergence however long it ran.

Fix. Those tests now ask for 10^{-11}, three orders above the measured floor and seven below the discretisation error of about 2×10^{-4} they actually measure. The table above is quoted in the test comment so the number is not mistaken for a guess.

Chapter 3

Concurrency

3.1 CONC-01: a destructor ordering hazard in the jthread pool

Symptom. Found by inspection during review of the shutdown path, before it could reproduce.

Root cause. Workers wait on a barrier and, once released, read an atomic `stopping_` flag before returning. `std::jthread` joins in its own destructor, and members are destroyed in reverse declaration order, which placed `stopping_` after the thread vector. A worker still reading the flag while it was being destroyed would be a use after free, in a window that would almost never reproduce and would be attributed to something else when it did.

Fix. Join every worker explicitly in the destructor body after releasing the barrier, before any member can be destroyed. Relying on the jthread destructor was tidier to read and wrong.

3.2 CUDA-02: the host was contracting to FMA and the device was not

Symptom. GPU Jacobi and GPU red black Gauss Seidel came out bit identical to the CPU, but GPU red black SOR differed by 1.6×10^{-18}, one bit in the last place.

Root cause. The device is compiled with `--fmad=false`, so it evaluates a multiply and an add separately. The host had no such restriction, and the SOR update has the shape $ab + cd$, which GCC is free to contract into a fused multiply add. It did. Jacobi has no such shape, which is why only SOR disagreed and why the discrepancy looked mysterious rather than systematic.

Options. Comparing SOR to a tolerance was rejected: the central claim of the library is that identical numerics run over every execution model, and "identical unless the compiler decided to contract" is a much weaker claim that happens to be invisible in the test output. Enabling FMA on the device was rejected because it would fix SOR and break Jacobi.

Fix. Turn contraction off on both sides, `-ffp-contract=off` on the host alongside `--fmad=false` on the device. The cost is nil in practice, these kernels being bandwidth bound, and the gain is that results no longer depend on whether a particular compiler on a particular target felt like contracting.

Verification. All three GPU sweeps are now bit identical to the CPU, and the device and host red black runs agree on iteration count exactly.

3.3 CUDA-01: kernels in a header become duplicate device stubs

Symptom. Multiple definition of `__device_stub__ZN8pnl_cuda11xpby_kernelEPKddPdii`, reported against a generated file with a name like `tmpxft_00280375_00000000-6_stream_probe.cudafe1.cpp`.

Root cause. The shared kernels were defined in a header. A `__global__` function in a header is emitted into every translation unit that includes it, along with its host side stub, so two `.cu` files including it produced two definitions of each. The error names a mangled stub in a generated file and gives no hint that a header is responsible.

Fix. The shared header now holds error handling, launch geometry and declarations only. Each kernel is defined in exactly one `.cu`, and anything needed across files goes through a plain launcher function.

3.4 CUDA-03: a device to device copy given a host pointer

Symptom. Every device solve failed immediately with `cudaMemcpy(...) failed: invalid argument`.

Root cause. A plain slip: the source should have been the device copy made two lines earlier, not the caller's host array. Recorded because the class of mistake is common at a C ABI boundary, where pointers carry no indication of which address space they belong to.

Chapter 4

Distributed memory

4.1 MPI-01: the iterate is distributed but the result must not be

Symptom. At two and four ranks, four separate test cases failed at once: order free solvers differed from serial by 3×10^{-4}, ordered solvers were not bit identical, dense systems differed by 0.998, and the discretisation error came out as 1 instead of 2×10^{-4}.

Root cause. One cause behind all four. During the iteration a rank needs only its own rows plus a halo, and the code was correct in that respect: residual norms and convergence detection were right on every rank. But the returned solution vector was left distributed, each rank holding only its own rows, and the tests compared the whole vector.

Options. Gathering after every sweep was rejected: it would dominate the communication measurement and would measure the wrong thing entirely, since the iteration genuinely does not need it.

Fix. A `synchronise` method on the problem, called once at the end of each solve. For the padded grid the owned rows form a single contiguous run of the array, so the gather needs no knowledge of the padding.

Verification. All MPI tests pass at one, two and four ranks, including bit identity against serial for the pipelined ordered sweeps.

4.2 MPI-02: block ranges are not row ranges

Symptom. Caught during review of the dense path rather than by a failing test.

Root cause. A rank's share of the *blocks* in a block method covers a different set of rows than its share of the *rows* in a point method, whenever the block count differs from the unknown count. A gather that assumed otherwise would have collected the wrong segments and corrupted the vector silently.

Fix. A general `gather_rows` primitive that collects each rank's range from the ranks themselves rather than assuming it, with the block derived range passed explicitly at the one call site where the two differ.

Chapter 5

Instrumentation and process

5.1 SWEEP-01: the resumable sweep was not resumable

Symptom. Restarting the sweep reported "38 already complete" and then immediately began re-running the first configuration, a four minute solve that was already recorded.

Root cause. The skip test was in the right loop but on the wrong side of the work. Rows were parsed from the binary's output and only then compared against the completed set, so every configuration was executed in full and the only thing skipped was writing the row. The sweep was resumable in the sense that it did not corrupt the summary, and in no other sense.

The reason it was written that way is that the identity tuple was taken from the emitted row, which does not exist until the run has happened. The circularity was invisible because the counters still reported plausible numbers.

Fix. Predict the identity from the declared configuration before anything is launched. The predictions that are not simply the declared value are the places this could go wrong again, so they are named in the code: the serial and CUDA backends report one worker whatever was requested, and the hybrid backend reports ranks times threads. A wrong prediction costs one redundant run, after which the real row makes the match exact, so the failure mode is waste rather than a missing measurement.

The commit is now probed from the binary rather than read from the first existing row, since a stale commit in the resume check would silently skip configurations whose code had changed.

Verification. Re-running a completed block reports 36 skipped in under a second, against roughly ten minutes of recomputation before.

5.2 SWEEP-02: two quadratic methods declared in a linear block

Symptom. Caught by arithmetic rather than by waiting. A sweep block intended for the methods whose iteration count grows like n listed symmetric Gauss Seidel and block Gauss Seidel at grid sizes 511 and 1023.

Root cause. Neither is $O(n)$. Symmetric Gauss Seidel at a relaxation factor of one, and line Gauss Seidel, are both $1 - O(h^2)$ and need of order n^2 iterations. At 1023 that is about a million sweeps over a million unknowns, hours per configuration, for a number the closed form already predicts.

Fix. The block now lists only optimally relaxed SOR, its red black form and conjugate gradient, with a comment giving the reason so the list is not helpfully extended later. A per configuration timeout was added to the sweep driver, so a future misdeclaration costs fifteen minutes rather than a night.

5.3 BUILD-01: CMake scans for modules that do not exist

Symptom. Every compile line carried `-fmodules-ts` and `-fdeps-format=p1689r5`, and the first build failed inside that machinery.

Root cause. From C++20 onward CMake scans each translation unit for module dependencies by default. This project is entirely header based and declares no modules, so the scan is pure cost and routes GCC through its experimental modules path.

Fix. `set(CMAKE_CXX_SCAN_FOR_MODULES OFF)`.

5.4 STYLE-01: the specification itself contained an em dash

Symptom. The dash checker's first run over the tree reported one violation, in the build specification, in the sentence asking for the personal report to be thorough.

Root cause. The document setting out the no dash ground rule contains one em dash in its own prose.

Fix. The specification is preserved for provenance with that one character replaced by a comma, and the substitution is recorded here rather than made silently, because the input to the build is part of the record.

Chapter 6

What the faults have in common

Five of the entries above produced plausible numbers rather than obvious failures: the eigenvector right hand side, the relaxation default, the resume logic, the tolerance below the arithmetic floor, and the fused multiply add discrepancy. Those are the ones worth generalising from.

Assert the shape, not just the outcome. The eigenvector fault was caught because a test asserted the iteration count should *grow* with the grid, not merely that the solver converged. A test checking only convergence would have passed forever while measuring nothing.

Exact comparisons find what tolerances hide. The one bit fused multiply add difference would have vanished into any reasonable tolerance. It was visible only because the equivalence suite asserts bit identity, and following it led to a real weakness in the project's central claim.

Measure the floor before choosing the target. Asking for accuracy below what the arithmetic can deliver produces a failure that looks like a regression in whatever component changed most recently.

Defaults should defer, not decide. A shared options structure whose defaults suit whichever component was written first will silently mis-configure the others.

Time the fast path, do not merely observe it. The resume logic reported correct counts while doing no work correctly. It survived until someone timed a restart instead of reading its output.